Formative 2 — Multimodal Data Preprocessing

User Identity & Product Recommendation System

Course: Machine Learning Pipeline — 2026 May Term (ALU)
Team members: Musana, Amaliza, Vestine, Emmanuel
GitHub repository: [FILL IN: repository URL]
System demonstration video: [FILL IN: video URL]
Date: 19 July 2026

---

1. Introduction and System Overview

This project implements a **multimodal identity-verification and product-recommendation system**. A user is authenticated in two sequential stages — facial recognition followed by voiceprint verification — and only after both succeed is a personalised product recommendation produced. If either stage fails, access is denied immediately and no recommendation is shown.

The system combines three independently trained models:

1. **Facial Recognition Model** — given a face image, identifies which enrolled member it belongs to.
2. **Voiceprint Verification Model** — given a voice recording, identifies which enrolled member is speaking.
3. **Product Recommendation Model** — given a customer's transaction and social-media features, predicts the product category they are most likely to purchase.

The end-to-end flow is: **face image → facial recognition → (if recognised) voice sample → voiceprint verification → (if recognised and identities agree) product recommendation.** Each biometric stage is a security checkpoint with an explicit *access-denied* path.

The repository is organised into four self-contained task folders (`task1_data_merge`, `task2_images`, `task3_audio`, `task4_models_and_demo`), each with its own notebook, so every stage of the pipeline can be run and reviewed independently.

---

2. Task 1 — Data Merge

2.1 Data sources

Two course-provided datasets were used:

- **`customer_transactions`** — 150 transactions across 75 unique customers (a customer may appear in more than one transaction). Key fields: `customer_id_legacy`, `transaction_id`, `purchase_amount`, `purchase_date`, `product_category`, `customer_rating`.
- **`customer_social_profiles`** — 155 rows across 84 unique customers, with *multiple rows per customer* (repeated platform interactions/reviews). Key fields: `customer_id_new`, `social_media_platform`, `engagement_score`, `purchase_interest_score`, `review_sentiment`.

2.2 Exploratory Data Analysis

Transactions. `purchase_amount` is spread fairly uniformly across roughly $51–$495 with **no outliers** under the IQR rule, so no values needed clipping. `customer_rating` was missing for 10 rows (~6.7%); all other fields were complete, and `transaction_id` contained no duplicates. There are five product categories: Sports, Electronics, Clothing, Groceries, and Books. The correlation between `purchase_amount` and `customer_rating` is close to zero — how much a customer spends does not predict how they rate the purchase.

Social profiles. `engagement_score` and `purchase_interest_score` are essentially uncorrelated, and purchase interest does not shift meaningfully with review sentiment. Five platforms (LinkedIn, Twitter, Facebook, TikTok, Instagram) and three sentiment classes (Positive/Neutral/Negative) are present, with no missing values.

Key insight. Social engagement on its own does not signal purchase intent, which is precisely why the transaction data is needed to supply a real purchase label for supervised learning.

Figure 2.1 — Transaction EDA: purchase-amount distribution, outliers, and transactions per category (real data).

Figure 2.2 — Social EDA: engagement, purchase-interest, and dominant-platform distributions (real data).

2.3 Data cleaning

- **`customer_rating`** (10 missing) was imputed using the **median rating within the same `product_category`**, rather than a single global median, since rating tendencies plausibly differ by category.
- **`customer_social_profiles`** contained 5 fully duplicated rows, which were dropped (155 → 150 rows).
- No outliers or duplicate transaction IDs required action.

2.4 Reconciling the join key and merge strategy

The two files identify customers differently: transactions use an integer `customer_id_legacy` (e.g. `151`) while social profiles use a string `customer_id_new` (e.g. `"A178"`). Every social ID was validated against the pattern `A\d+`, then the `"A"` prefix was stripped and the value cast to an integer, recovering a common numeric `customer_id` present in both files.

An **inner join** was chosen deliberately. The recommendation model needs both a **label** (which only exists for customers with transactions) and **features** (which only exist for customers with social activity). A customer missing either side cannot be used for supervised training, so retaining only the overlap is correct; a left join would leave feature-less rows requiring heavy imputation of social data that simply does not exist.

Because the social table has **multiple rows per customer**, a direct merge on `customer_id` would *fan out* every transaction by that customer's number of social rows and corrupt the row-level meaning of the dataset. Social data was therefore **aggregated to one row per customer first** — mean engagement, purchase-interest, and (numeric) sentiment scores; dominant platform and sentiment; platform diversity; and interaction count — and then merged into the transaction-level table so each transaction inherits its customer's aggregated social profile.

2.5 Post-merge validation

Three checks confirmed the merge behaved correctly:

1. **Row-count check** — the merged row count equalled the number of transactions belonging to overlapping customers exactly, confirming no fan-out and no unexpected row loss.
2. **Null check** — zero nulls were introduced by the merge.
3. **Spot check** — one customer's aggregated `avg_engagement_score` was recomputed by hand and matched the merged value.

2.6 Feature engineering

Beyond the aggregated social features (which are themselves cross-source engineered features), the following were derived on the merged table: `purchase_month` (from `purchase_date`), `is_high_value_purchase` (above-median spend flag), and integer encodings `product_category_code` and `dominant_platform_code`.

Result: the final merged dataset is **117 rows × 17 columns**, spanning **61 unique customers** and the five product categories (Sports 28, Electronics 27, Clothing 22, Groceries 20, Books 20). It is saved as `merged_customer_dataset.csv`.

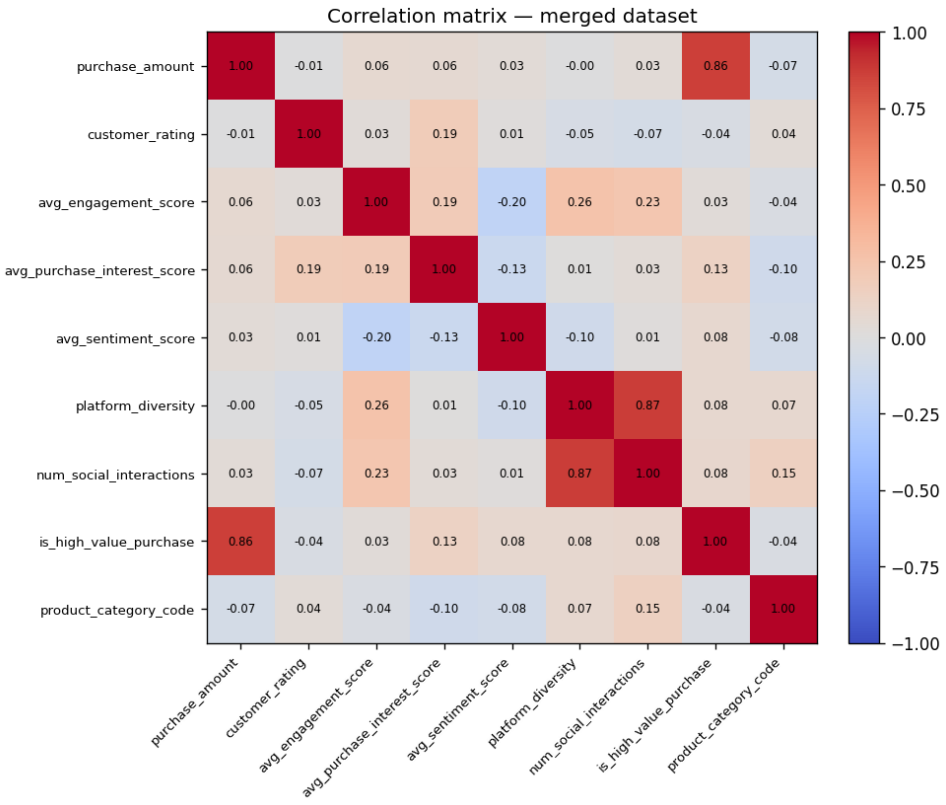

Figure 2.3 — Correlation matrix of transaction and social features on the merged dataset.

3. Task 2 — Image Data: Collection, Augmentation, Feature Extraction

> **Data status:** the pipeline below is complete and validated. The repository currently runs it against a synthetic placeholder identity so the code can be verified end-to-end; **replace `images/<member>/` with each teammate's real photos and re-run before submission.**

3.1 Collection convention

Each member contributes **three facial expressions — neutral, smiling, and surprised** — stored as `images/<member_name>/{neutral,smiling,surprised}.jpg`. The pipeline auto-discovers whatever member folders exist, so no code changes are needed when real photos are added.

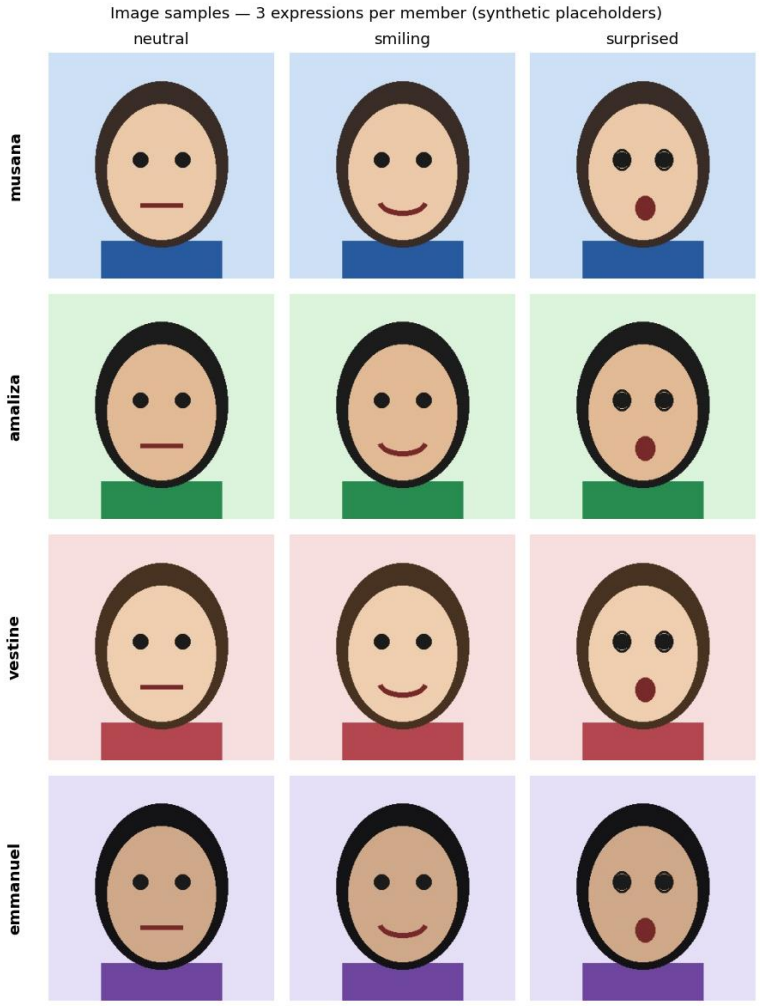

Figure 3.1 — Three expressions per member (synthetic placeholders).

3.2 Augmentation

Three augmentations are applied to every image (the rubric requires at least two): **rotation** (15°), **horizontal flip**, and **grayscale conversion**. Each original photo plus its

three augmented versions is carried forward, so the feature set captures pose and lighting variation rather than a single static shot.

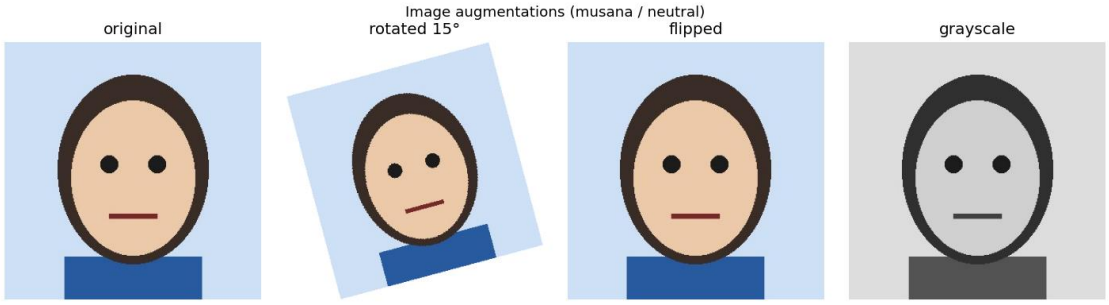

Figure 3.2 — Original image and its three augmentations.

3.3 Feature extraction

For every (member, expression, augmentation) combination, an **84-dimensional feature vector** is extracted and written as one row of `image_features.csv`:

- **Color histogram** — 16 bins per RGB channel (48 values) — overall colour/lighting distribution.
- **HOG (Histogram of Oriented Gradients)** — edge/shape structure, summarised to a fixed-length 32-value histogram so the vector length is stable across image sizes.
- **Grayscale pixel statistics** — mean, standard deviation, min, and max brightness (4 values).

A row-count validation confirms the table contains exactly *images × augmentations* rows with no nulls. With four members this yields 4 × 3 expressions × 4 variants = **48 rows**.

---

4. Task 3 — Sound Data: Collection, Visualisation, Augmentation, Feature Extraction

> **Data status:** as with Task 2, the pipeline is complete and validated on a synthetic placeholder voice; **replace `audio/<member>/` with each teammate's real recordings and re-run before submission.**

4.1 Collection convention

Each member records **two phrases** — `phrase1.wav` = *"Yes, approve"* and `phrase2.wav` = *"Confirm transaction"* — stored as `audio/<member_name>/`.

4.2 Visualisation and interpretation

Each recording is plotted as a **waveform** and a **spectrogram**. Spectrograms reveal per-speaker **formant** structure — horizontal energy bands whose positions are set by vocal-tract length — which is exactly what allows a voiceprint model to separate speakers saying the *same* words.

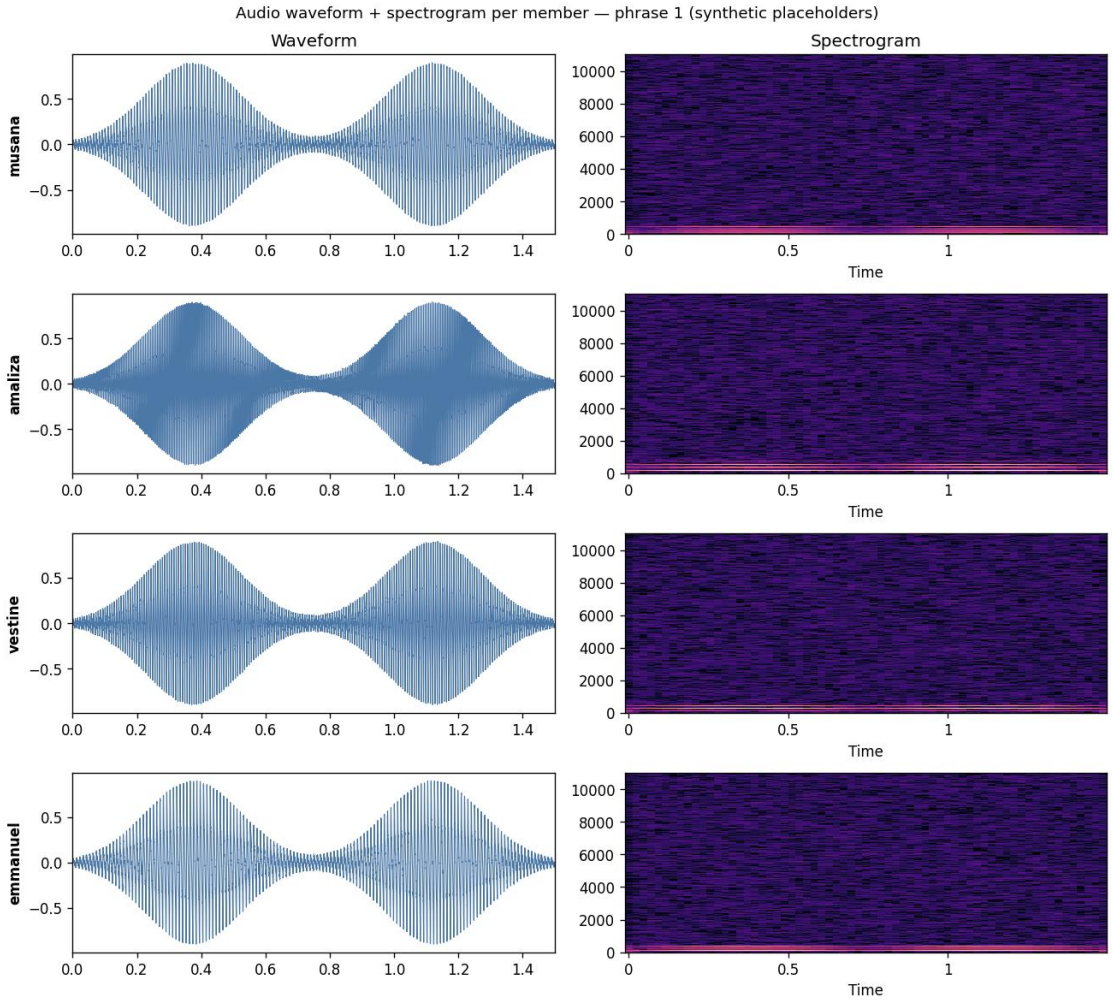

Figure 4.1 — Waveform and spectrogram per member, phrase 1 (synthetic placeholders).

4.3 Augmentation

Three augmentations are applied to every clip (again exceeding the required two): **pitch shift** (+3 semitones), **time stretch** (1.2×), and **additive background noise**, capturing vocal-pitch, pacing, and noisy-environment variation.

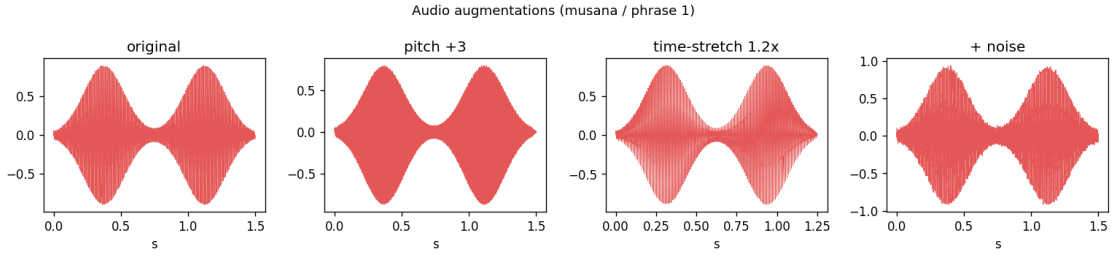

Figure 4.2 — Original clip and its three augmentations.

4.4 Feature extraction

For every (member, phrase, augmentation) combination, a **31-dimensional feature vector** is written as one row of `audio_features.csv`:

- **MFCCs** — 13 coefficients, mean and standard deviation pooled over time (26 values) — the standard timbre/voiceprint descriptor.
- **Spectral roll-off** — mean and standard deviation (2 values) — a brightness indicator.
- **RMS energy** — mean and standard deviation (2 values) — loudness over time.
- **Duration** (1 value).

A row-count validation confirms *recordings × augmentations* rows with no nulls (with four members: 4 × 2 phrases × 4 variants = **32 rows**).

---

5. Task 4 — Model Creation and Evaluation

All three models use algorithms from the assignment's suggested set (Random Forest / Logistic Regression) and are evaluated with **accuracy, macro-F1, and log-loss**.

5.1 Facial Recognition Model

A **Random Forest** (200 trees) is trained on the 84 image features, with the member name as the label, using a stratified 70/30 train/test split. The synthetic `UNAUTHORIZED_ATTEMPT` sample is **held out of training entirely** — an "unknown" identity must never be a training class, or the model would learn to recognise one specific impostor rather than to reject *any* stranger.

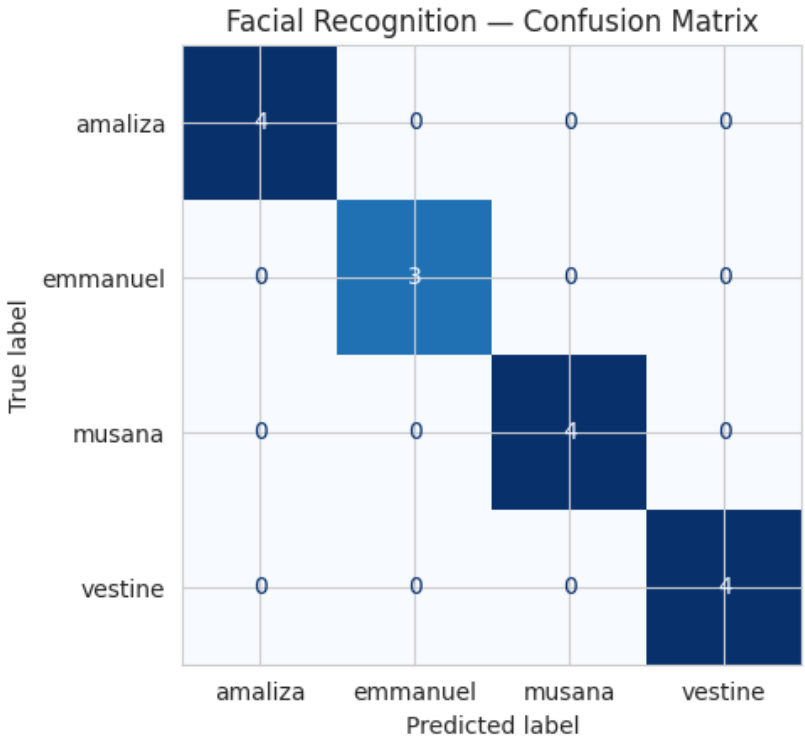

Figure 5.1 — Facial recognition confusion matrix (placeholder data).

5.2 Voiceprint Verification Model

The same structure is applied to the 31 audio features: a Random Forest trained on authorized speakers, with `UNAUTHORIZED_ATTEMPT` held out as an inference-time probe.

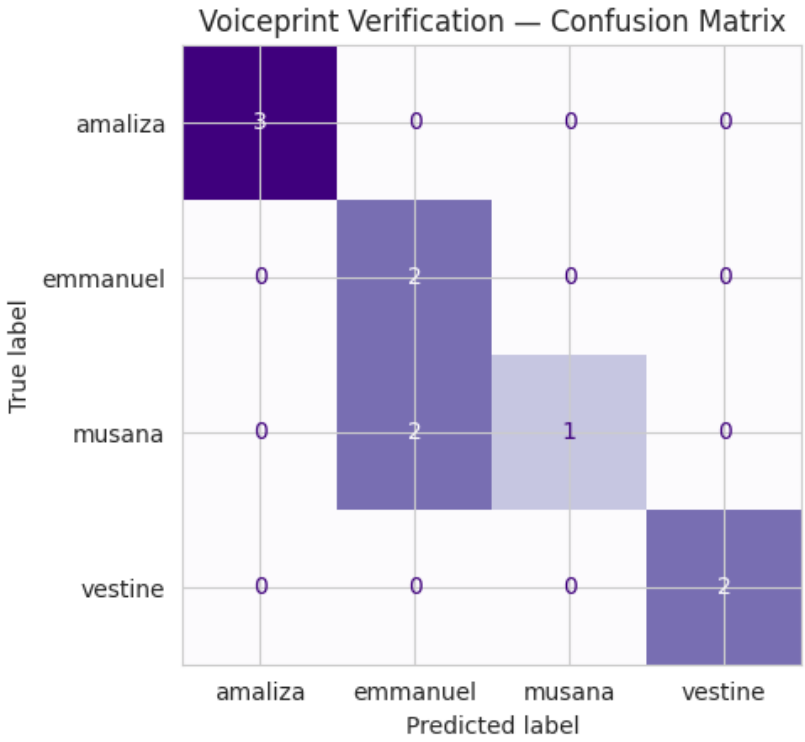

Figure 5.2 — Voiceprint verification confusion matrix (placeholder data).

5.3 Product Recommendation Model

Two candidates — **Random Forest** and **Logistic Regression** — are trained on the **real** merged dataset (transactions + aggregated social features) to predict `product_category`, using a stratified 75/25 split, and the better model by accuracy is saved. Random Forest was selected.

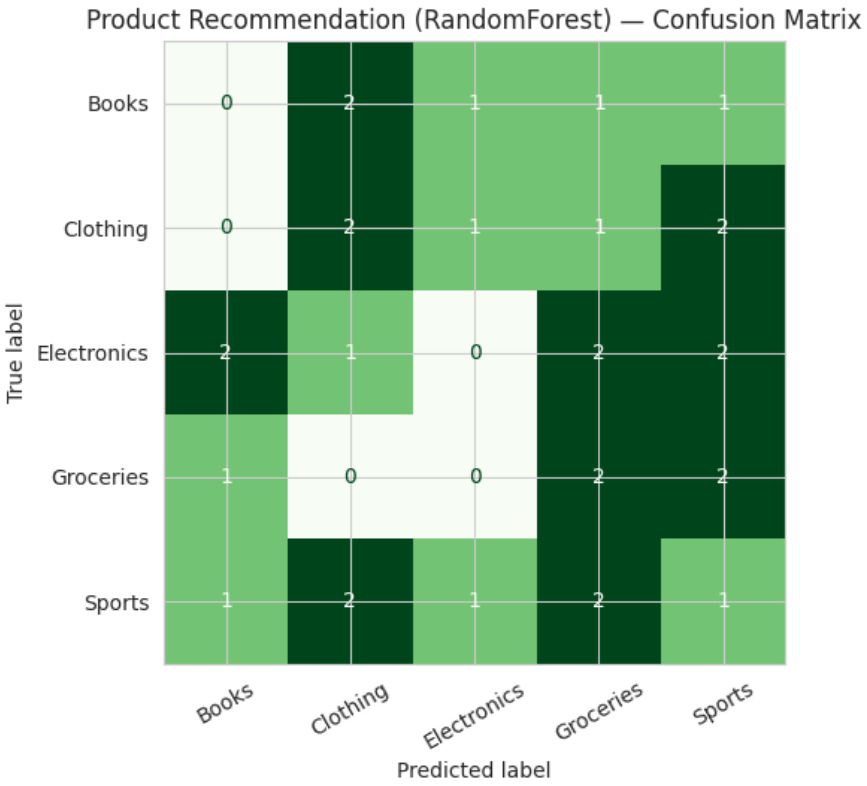

Figure 5.3 — Product recommendation confusion matrix (real data).

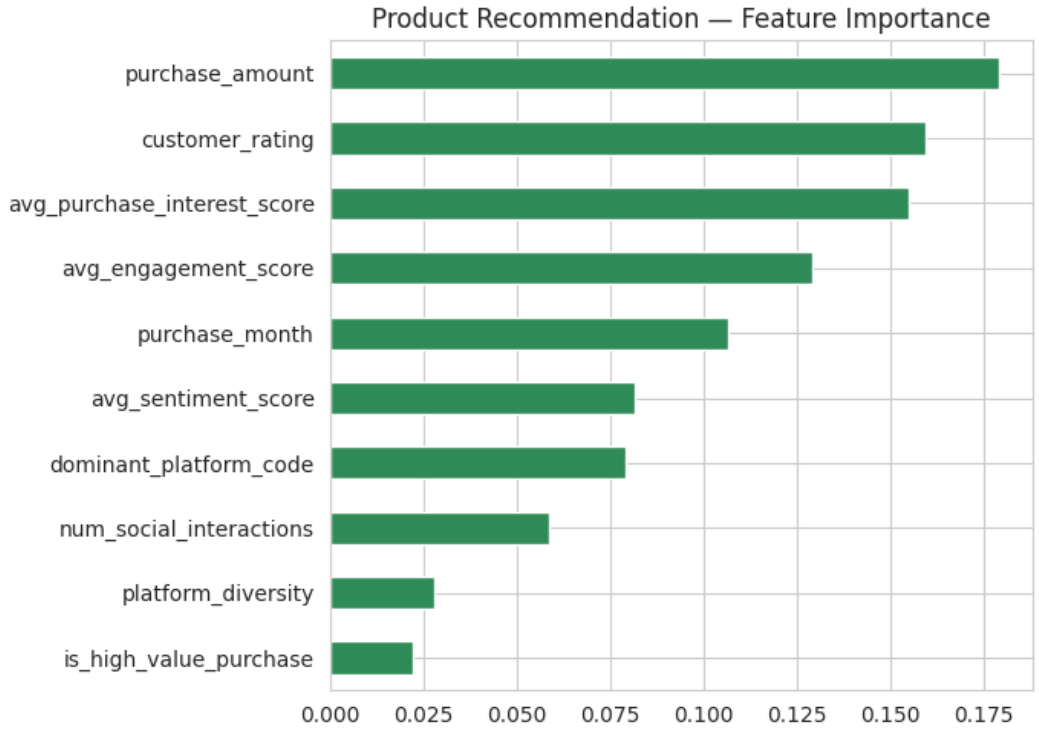

Figure 5.4 — Product model feature importances (real data).

5.4 Results

Model	Training data	Accuracy	Macro-F1	Log-loss
Facial Recognition (Random Forest)	placeholder — *replace*	1.00	1.00	0.141
Voiceprint Verification (Random Forest)	placeholder — *replace*	0.80	0.79	0.625
Product Recommendation (Random Forest)	real merged dataset	0.167	0.15	1.865

Reading these numbers honestly:

- The facial and voiceprint scores are measured on **synthetic placeholder data** (a cartoon drawing and a generated tone), so they reflect how trivially separable that placeholder data is — **not** real-world biometric performance. **Re-run Task 4 on your real photos/recordings and replace this row**; expect harder, more realistic numbers.
- The product-recommendation accuracy of **0.167 is roughly chance for a 5-class problem**. This is a genuine finding, not a bug: as the Task 1 EDA showed, the available social/transaction features correlate only weakly with purchased category, and with 117 rows the per-class test support is small. More transaction history per customer, or richer features, would be needed for real predictive power. Stating this limitation clearly is more valuable than reporting an inflated number.

5.5 Multimodal decision logic

Authentication combines several checks rather than a bare prediction:

1. **Two sequential gates.** The face must be recognised before the voice is even requested; failure at either gate denies access immediately.
2. **Open-set rejection at each gate.** A standard classifier always outputs *some* class, so identity prediction alone cannot reject a stranger. Each biometric model therefore accepts an identity only if it passes **both** a **confidence threshold** (top predicted probability ≥ 0.5) **and** a **centroid-distance threshold** (distance to that identity's training centroid within the 95th percentile of that identity's own training spread).
3. **Cross-modal consistency.** The identity predicted from the face must match the identity predicted from the voice; otherwise access is denied — defeating an attack that pairs a stolen photo with a different person's voice.

An honest note on how "same person" is enforced. The facial and voiceprint models are two *independent* classifiers that share no features; the face model never sees audio and vice-versa. What links a member's face to their voice is the **enrolment label** — the folder name under which their photos and recordings are stored — combined with the string-equality check in step 3. The system does not learn a joint face–voice biometric; it checks that two unimodal classifiers independently produced the same enrolled name. This is a

valid design at this scale, but it means correct, consistent folder naming per teammate (identical name in `images/` and `audio/`) is essential, and a true single-identity biometric would require a fused cross-modal embedding (beyond this assignment's scope).

---

6. Task 5 — System Simulation

The command-line application `cli_app.py` implements the full flow. A shared module, `feature_extraction.py`, is used at **both** training time and inference time, so a live photo or recording is turned into a feature vector in exactly the same way as the training data.

The `run_demo.sh` script runs the three required scenarios:

1. **Authorized transaction** — a recognised face clears gate 1, a matching voice clears gate 2, the identities agree, `ACCESS GRANTED` is printed, and a product recommendation is produced.
2. **Unauthorized attempt (unrecognised face)** — rejected at gate 1; the voice is never requested.
3. **Unauthorized attempt (valid face, unrecognised voice)** — clears gate 1 but is rejected at gate 2, demonstrating that a face alone is insufficient.

The captured output of `run_demo.sh` (`demo_transcript.txt`):

```
########## DEMO 1: VALID / AUTHORIZED TRANSACTION ##########
============================================================
STEP 1: Facial recognition
  Input: images/musana/smiling.jpg
  Result: {'authorized': True, 'identity': 'musana', 'confidence': 0.975,
'distance': 4.048734144598118}
  Face recognized as: musana

STEP 2: Voiceprint verification
  Input: audio/musana/phrase2.wav
  Result: {'authorized': True, 'identity': 'musana', 'confidence': 0.805,
'distance': 1418.9923523934447}
  Voice recognized as: musana

  ACCESS GRANTED for musana.

STEP 3: Product recommendation
  (no record for customer_id=178, sampling one instead)
  Customer ID used for input features: 146
  Predicted product category: Groceries  (confidence: 0.26)
============================================================

########## DEMO 2: UNAUTHORIZED ATTEMPT -- unrecognized face ##########
============================================================
STEP 1: Facial recognition
```

```
  Input: images/UNAUTHORIZED_ATTEMPT/neutral.jpg
  Result: {'authorized': False, 'identity': None, 'confidence': 0.29,
'distance': 43.53876352253075}

  ACCESS DENIED — face not recognized.

########## DEMO 3: UNAUTHORIZED ATTEMPT -- unrecognized voice (face alone is
not enough) ##########
============================================================
STEP 1: Facial recognition
  Input: images/amaliza/neutral.jpg
  Result: {'authorized': True, 'identity': 'amaliza', 'confidence': 0.995,
'distance': 8.596990254800788}
  Face recognized as: amaliza

STEP 2: Voiceprint verification
  Input: audio/UNAUTHORIZED_ATTEMPT/phrase1.wav
  Result: {'authorized': False, 'identity': None, 'confidence': 0.645,
'distance': 3866.714923833067}

  ACCESS DENIED — voice not recognized.
```

System-demonstration video: [FILL IN: video URL]

---

7. Design Decisions and Limitations

- **Placeholder biometric data.** All face and voice results in §5.4 are on synthetic placeholders and are pipeline sanity checks, not real performance; they will change once real captures are added.
- **Weak product-recommendation signal.** Near-chance accuracy reflects genuinely weak correlation between the available features and purchased category, not a coding error.
- **Two identity spaces.** The biometric models identify *team members*, while the product model uses `customer_id` from the *e-commerce* datasets — these are different populations that the assignment's sources do not naturally link. The recommendation step therefore looks up (or samples) a customer record independently of the authenticated member; this is a deliberate simplification, stated explicitly.
- **Identity linkage by label, not joint biometric** (see §5.5).
- **Small test sets** mean all metrics are indicative rather than robust estimates.

---

8. Reproducibility and Repository Structure

Dependencies are pinned in `requirements.txt`. The notebooks are run in order: `task1_data_merge` → `task2_image_pipeline` → `task3_audio_pipeline` → `task4_models`, then `run_demo.sh` for the simulation. The course datasets (`customer_transactions.xlsx`,

`customer_social_profiles.xlsx`) should be placed in `task1_data_merge/` and committed to the repository (the loader resolves them from that folder, so the notebook runs top-to-bottom from a fresh clone).

The repository contains all required deliverables: the raw datasets, the merged dataset with engineered features, `image_features.csv` and `audio_features.csv`, Python scripts for all three models plus the CLI app, and the four Jupyter notebooks.

---

9. Team Contributions

Suggested division based on the four tasks — replace with what each member actually did.

Member	Contribution
Musana	Task 1 — dataset cleaning, merge, EDA, and feature engineering (§2)
Amaliza	Task 2 — image collection, augmentation, and feature extraction; facial recognition model (§3, §5.1)
Vestine	Task 3 — audio collection, augmentation, and feature extraction; voiceprint model (§4, §5.2)
Emmanuel	Tasks 4–5 — product recommendation model, CLI app, and system demonstration (§5.3, §6)

---

10. Conclusion

The system delivers a complete multimodal pipeline: two data sources are cleaned, reconciled, and merged with validated join logic and engineered cross-source features; image and audio data are collected, augmented, and reduced to compact feature vectors; three models are trained and evaluated; and a command-line application enforces a two-gate, cross-modal authentication flow before serving a recommendation. The most important honest takeaways are that the biometric results must be regenerated on real team data before submission, and that the product-recommendation task is genuinely hard given the weak signal in the available features — a limitation identified directly from the exploratory analysis.